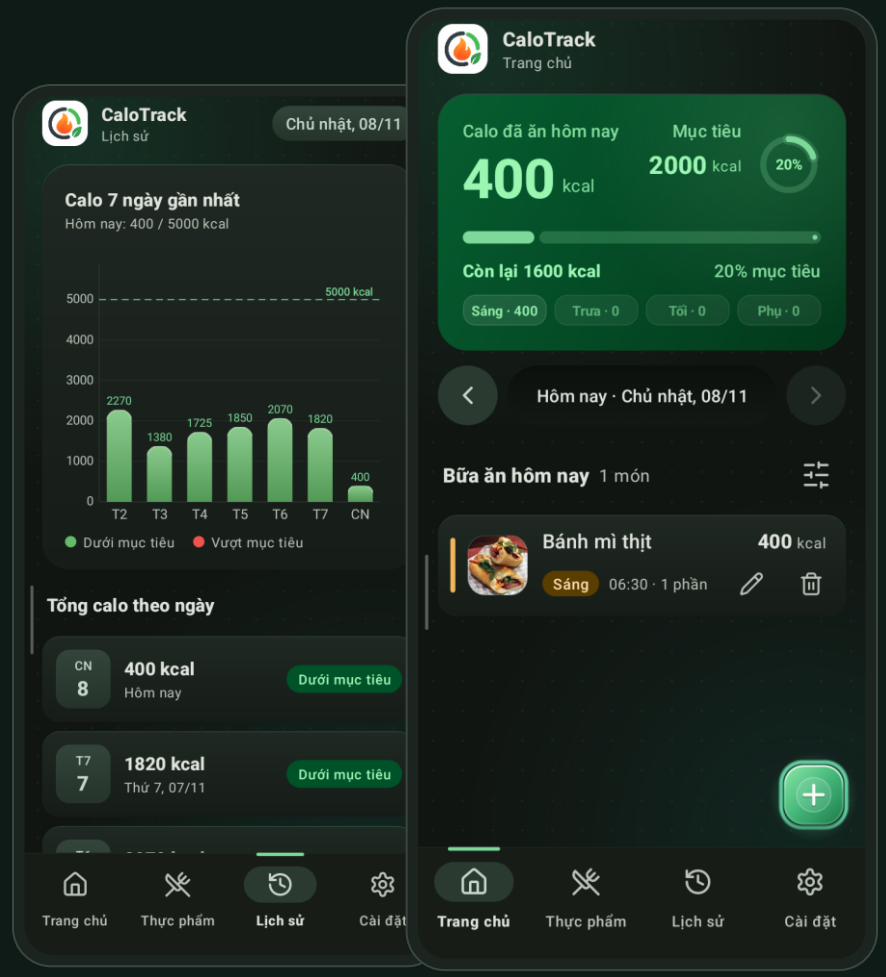
BÁO CÁO BÀI TẬP LỚN · ĐỀ 10

Ứng dụng đếm calo CaloTrack

Ứng dụng Android giúp người dùng theo dõi lượng calo nạp vào hằng ngày, quản lý bữa ăn và so sánh với mục tiêu · xây dựng bằng Kotlin, Room (SQLite) và Material 3.



Nhóm [SỐ NHÓM]

Trường/Khoa: [TÊN TRƯỜNG/KHOA]
Giảng viên hướng dẫn: [GIẢNG VIÊN HƯỚNG DẪN]
Thành viên: [DANH SÁCH THÀNH VIÊN]

Tổng quan đề tài

CaloTrack là ứng dụng Android chạy độc lập trên thiết bị: người dùng ghi lại bữa ăn theo loại bữa và khẩu phần, ứng dụng tự động tính calo, đối chiếu với mục tiêu hằng ngày và trực quan hóa lịch sử theo ngày bằng danh sách tổng hợp cùng biểu đồ cột 7 ngày.

89

món ăn Việt Nam kèm ảnh thật có sẵn khi cài đặt

4

màn hình chính: Trang chủ, Thực phẩm, Lịch sử, Cài đặt

2

bảng dữ liệu SQLite quản lý qua Room (foods & meals)

3

thiết lập lưu SharedPreferences: mục tiêu, đơn vị, sáng/tối

Nền tảng: Android 7.0 (API 24) trở lên · targetSdk 34 · hoạt động 100% offline · không cần đăng ký tài khoản hay kết nối Internet.

Lý do chọn đề tài

01

Nhu cầu theo dõi dinh dưỡng thực tế

Giảm cân, tăng cơ hay duy trì chế độ ăn lành mạnh đều bắt đầu từ việc kiểm soát lượng calo nạp vào mỗi ngày.

02

Tối ưu cho ẩm thực Việt Nam

Các ứng dụng quốc tế thiếu món Việt; CaloTrack tích hợp sẵn 89 món quen thuộc (Phở, Bún chả, Cơm tấm, Bánh mì) kèm ảnh trực quan.

03

Bao trọn kiến thức lập trình Android

Kết hợp đầy đủ Activity/Fragment, RecyclerView, kiến trúc MVVM (ViewModel - LiveData - Flow), Room SQLite, SharedPreferences và vẽ Canvas.

04

Dữ liệu riêng tư, phản hồi tức thì

Toàn bộ dữ liệu nằm trong vùng nhớ nội bộ của ứng dụng (/data/data/com.ndtien.calotrack), mở app dùng ngay không cần đăng nhập.

Mục tiêu và phạm vi

Mục tiêu

- Xây dựng ứng dụng Android chạy mượt, ổn định từ Android 7.0 (API 24 - 34)
- Đáp ứng trọn vẹn 9 yêu cầu chức năng bắt buộc của Đề 10
- Tổ chức mã nguồn tách lớp chuẩn MVVM: View · ViewModel · Room DAO (SQLite)
- Lưu trữ bền vững bằng SQLite cho dữ liệu và SharedPreferences cho cài đặt
- Thiết kế giao diện Material 3 Soft-Tech 3D hỗ trợ cả chế độ Sáng và Tối

Phạm vi

- **Trong phạm vi:** Quản lý thực phẩm, ghi nhận bữa ăn, tự tính calo, đổi màu trạng thái, biểu đồ 7 ngày, cài đặt
- **Mở rộng thêm:** Bộ 89 ảnh món ăn thật, lọc đa tiêu chí, vuốt để xóa kèm Hoàn tác (Undo), điều hướng xem ngày cũ
- **Ngoài phạm vi:** Đồng bộ đám mây, quét mã vạch, kết nối đồng hồ thông minh, tách chi tiết Đạm / Béo / Tinh bột

Yêu cầu chức năng (Đề 10)

01 CSDL thực phẩm có sẵn

89 món ăn nạp sẵn vào SQLite ngay khi khởi tạo; cho phép thêm, sửa, xóa thực phẩm

02 Thêm bữa ăn hằng ngày

Sáng / Trưa / Tối / Phụ; chọn từ danh sách hoặc nhập nhanh; tùy chọn ngày ghi nhận

03 Tự động tính calo

Tổng calo = calo món x khẩu phần (0,5 · 1,0 · 1,5…); cho phép sửa, xóa bữa ăn đã nhập

04 Danh sách bữa ăn theo giờ

Sắp xếp theo thứ tự thời gian trong ngày; hiển thị ảnh món, tên món, khẩu phần, calo

05 Tổng quan & đổi màu

Hiển thị tổng đã ăn, mục tiêu, còn lại hoặc vượt; đổi màu Xanh khi dưới, Đỏ khi vượt

06 Lịch sử & biểu đồ cột

Xem lịch sử tổng calo mỗi ngày dạng danh sách; biểu đồ cột hiển thị calo 7 ngày trong tuần

07 Đặt mục tiêu calo

Cho phép người dùng đặt mục tiêu 500 - 10.000 kcal/ngày, lưu bằng SharedPreferences

08 Tìm kiếm thực phẩm

Tìm kiếm không dấu theo thời gian thực kết hợp lọc theo nhóm món và khoảng calo

09 Lưu trữ cục bộ

SQLite (Room) cho thực phẩm & lịch sử bữa ăn; SharedPreferences cho mục tiêu, đơn vị, sáng/tối

Đối tượng sử dụng và luồng thao tác

01 · GHI NHẬN

Hằng ngày

- Chọn món từ danh mục 89 món sẵn có
- Nhập nhanh món tự nấu ngoài danh mục
- Tăng/giảm khẩu phần theo bước 0,5
- Sửa, vuốt xóa hoặc Hoàn tác (Undo)

02 · THEO DÕI

Trong ngày & tuần

- Xem tổng calo đã ăn so với mục tiêu
- 4 viên thuốc thống kê Sáng·Trưa·Tối·Phụ
- Thẻ Hero đổi màu Xanh / Đỏ trực quan
- Đối chiếu biểu đồ cột 7 ngày gần nhất

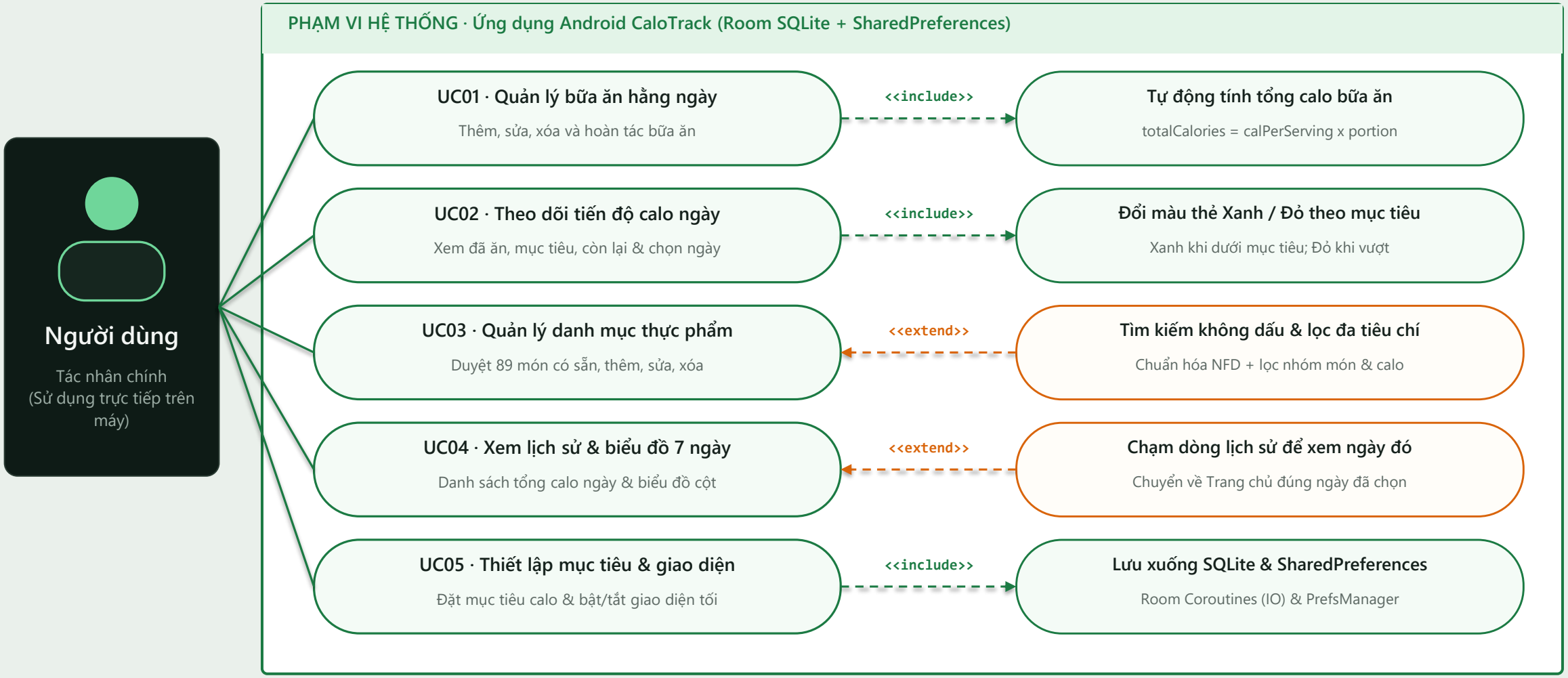
03 · TÙY CHỈNH

Cá nhân hóa

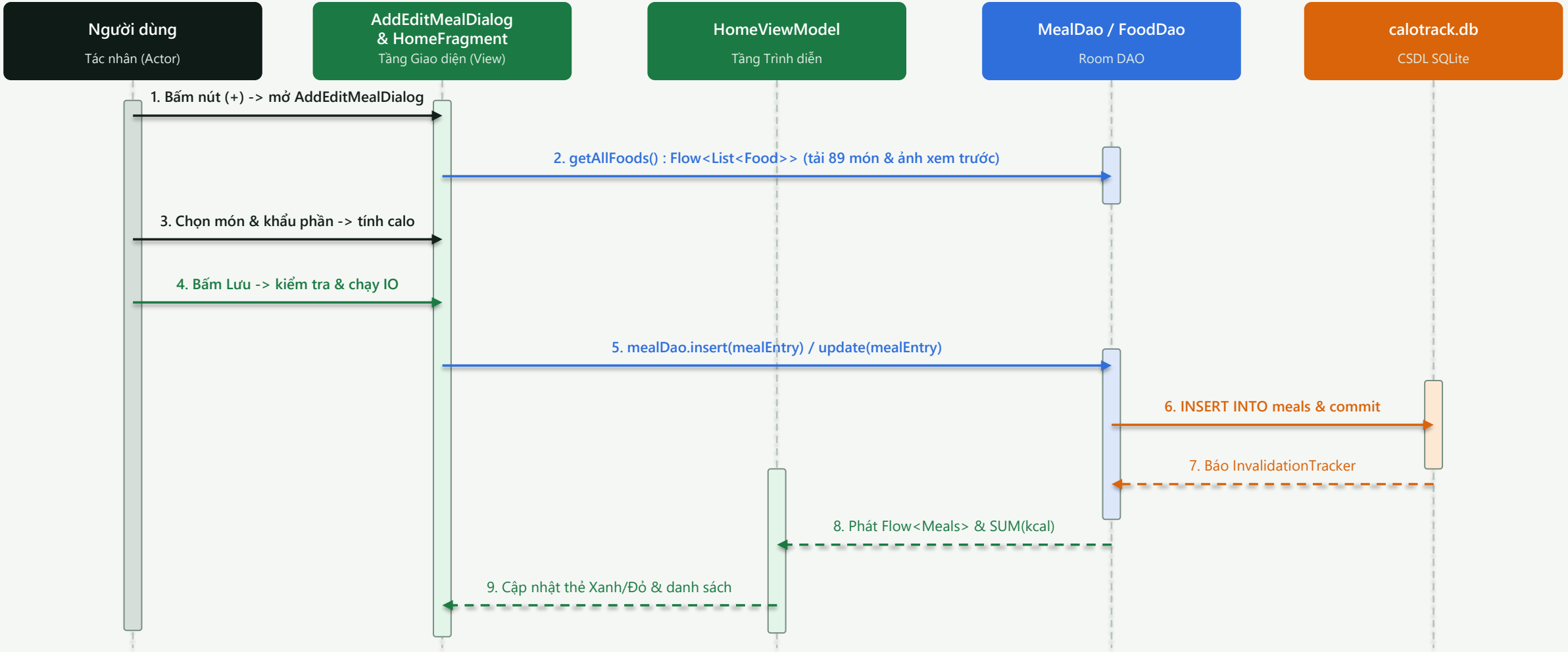
- Đặt mục tiêu calo (500 - 10.000 kcal)
- Thêm, sửa, xóa thực phẩm cá nhân
- Tìm kiếm không dấu & lọc theo nhóm
- Chuyển đổi giao diện Sáng / Tối mượt mà

Đối tượng sử dụng: Cá nhân (sinh viên, người ăn kiêng, người tập thể hình) cần công cụ theo dõi calo gọn nhẹ. Dữ liệu lưu cục bộ tại /data/data/com.ndtien.calotrack/.

Sơ đồ ca sử dụng (UML Use Case Diagram)



Sơ đồ tuần tự (Sequence Diagram): Thêm bữa ăn & cập nhật UI



Kiến trúc phân lớp MVVM tổng thể

Tầng Giao diện (View)

MainActivity + 4 Fragment, BottomSheet, Uiverse Custom Views, ViewBinding

Sự kiện / Observe

Tầng ViewModel

HomeViewModel & HistoryViewModel · LiveData, switchMap, MediatorLiveData

Truy vấn / Flow

Tầng Dữ liệu (Room SQLite)

FoodDao, MealDao trả về luồng Flow · calotrack.db

SharedPreferences

PrefsManager: mục tiêu calo, giao diện tối, đơn vị

LUỒNG GHI DỮ LIỆU (COROUTINES)

Người dùng bấm Lưu -> Dialog kiểm tra dữ liệu -> lifecycleScope.launch(Dispatchers.IO) gọi DAO insert / update / delete trên luồng nền.

LUỒNG ĐỌC PHẢN ỨNG (REACTIVE FLOW)

DAO trả về Flow<T> -> chuyển sang asLiveData() -> Fragment observe; mọi thay đổi trong SQLite đều tự động cập nhật giao diện.

Công nghệ sử dụng

Tầng	Công nghệ	Vai trò trong CaloTrack
Ngôn ngữ	Kotlin 1.9.22 · JDK 17	100% mã nguồn Kotlin; Coroutines & Flow xử lý bất đồng bộ
Hệ thống Build	Gradle 8.6 · AGP 8.3.2 · KSP	Biên dịch ứng dụng và sinh mã nguồn Room DAO lúc build
Nền tảng SDK	compileSdk 34 · minSdk 24	Tương thích từ Android 7.0 (Nougat) đến Android 14
Giao diện	Material 3 (1.12.0) · ViewBinding	BottomNavigation, Card, Chip, BottomSheet, ánh xạ View an toàn kiểu
Đồ họa tùy biến	Android Canvas API · LruCache	UiverseCalorieRingView, UiverseOrbFabView (FAB 3D), RoundedGradientBarRenderer
Kiến trúc	AndroidX Lifecycle 2.8.0 (MVVM)	ViewModel, LiveData, switchMap và MediatorLiveData
Cơ sở dữ liệu	Room 2.6.1 trên nền SQLite	Quản lý 2 bảng foods và meals; kiểm tra cú pháp SQL lúc biên dịch
Lưu thiết lập	SharedPreferences (calotrack_prefs)	Lưu mục tiêu calo (500-10.000 kcal), đơn vị năng lượng và chế độ tối
Biểu đồ	MPAndroidChart v3.1.0 (JitPack)	Biểu đồ cột calo 7 ngày gần nhất kèm đường giới hạn mục tiêu

Cấu trúc mã nguồn

```
app/src/main/
├─ assets/foods/                89 ảnh món ăn thật (.jpg)
├─ java/com/ndtien/calotrack/
│  ├─ MainActivity.kt           BottomNav + quản lý 4 Fragment
│  ├─ UiverseCalorieRingView.kt Vòng tròn % calo trên thẻ Hero
│  ├─ UiverseOrbFabView.kt      Nút (+) 3D Ngọc Lục Bảo + Sonar
│  ├─ data/                     Food · MealEntry · DAOs · DB
│  ├─ ui/home/                  HomeFragment · HomeViewModel
│  ├─ ui/meal/                  AddEditMealDialog
│  ├─ ui/food/                  FoodList · Adapter · Bộ lọc
│  ├─ ui/history/               HistoryFragment · BarRenderer
│  ├─ ui/settings/              SettingsFragment
│  └─ util/                     DateUtils · Prefs · ImageLoader
└─ res/                         Layouts · Drawables · Night Theme
```

data · Tầng lưu trữ Room

Food và MealEntry là @Entity; FoodDao và MealDao chứa câu truy vấn SQL; SeedData nạp 89 món Việt.

ui · Chia theo tính năng

Mỗi gói màn hình gồm Fragment, ListAdapter (DiffUtil) và ViewModel / BottomSheetDialog riêng biệt.

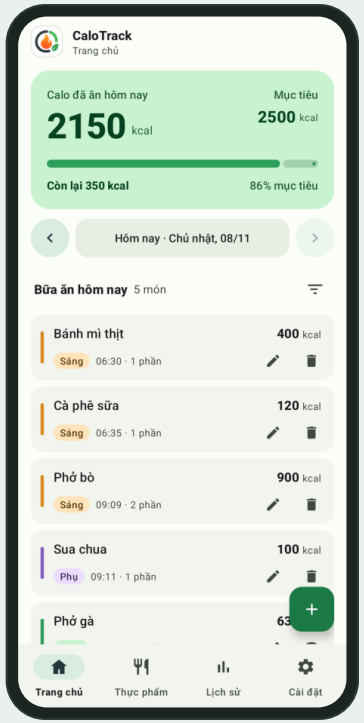
Custom Canvas Views 3D

UiverseCalorieRingView, UiverseOrbFabView và RoundedGradientBarRenderer vẽ đồ họa 60fps.

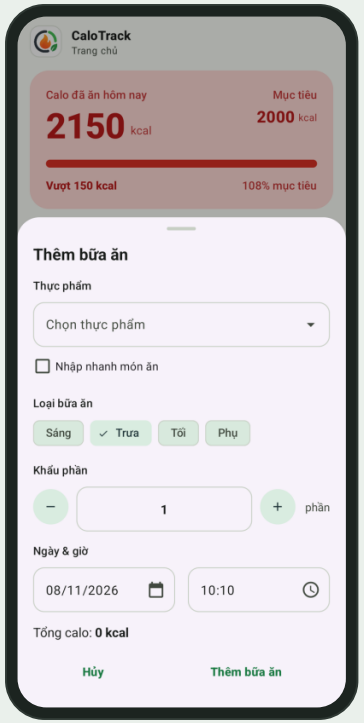
assets & res · Đa giao diện

89 ảnh món ăn 160x160 tối ưu qua LruCache; hỗ trợ đầy đủ giao diện Sáng và Tối (values-night).

Xây dựng giao diện người dùng (Sáng & Tối)



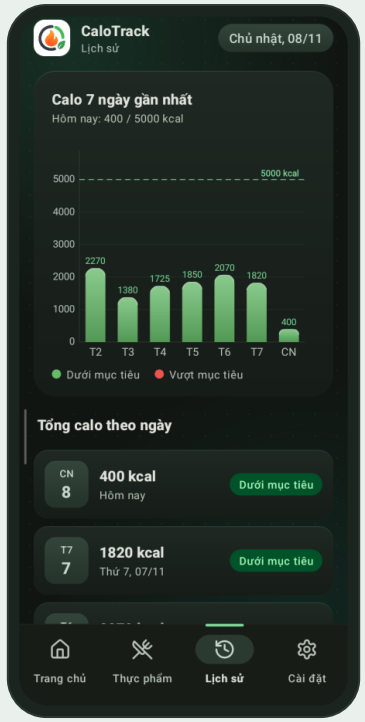
Trang chủ (Sáng)



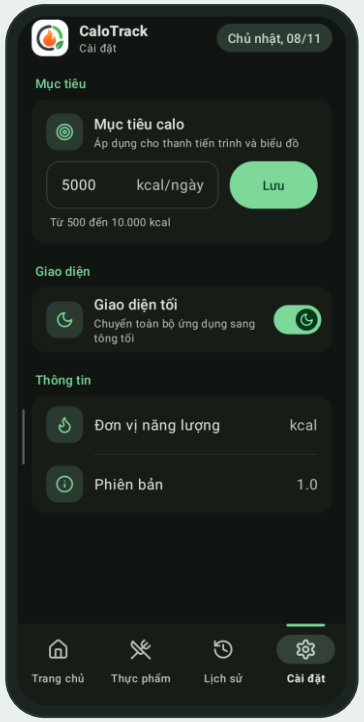
Thêm bữa ăn



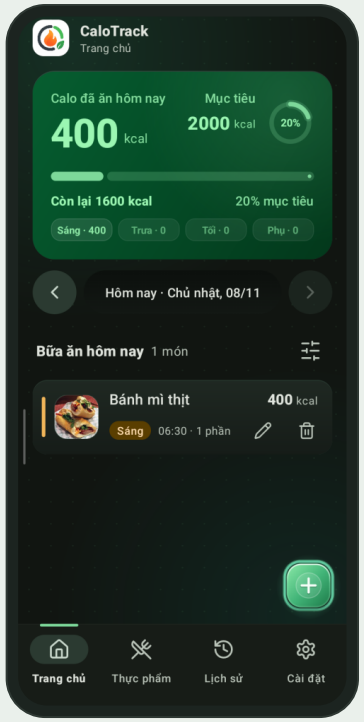
Thực phẩm (89 ảnh)



Lịch sử 7 ngày



Cài đặt



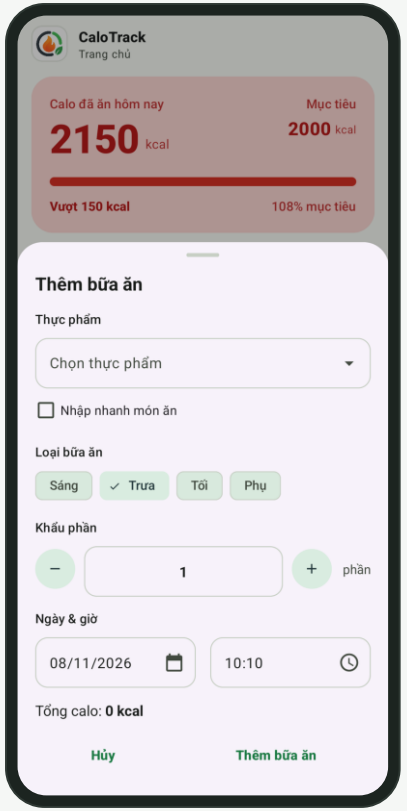
Trang chủ (Tối 3D)

Xử lý chức năng: thêm và tính calo bữa ăn

1. Tự chọn loại bữa theo giờ hiện tại (Sáng, Trưa, Tối, Phụ)
2. Chọn món qua ô gợi ý (AutoComplete kèm ảnh) hoặc tick Nhập nhanh
3. Tăng/giảm khẩu phần bằng nút - / + (bước 0,5; nhận cả dấu phẩy)
4. Xem trước tức thì: totalCalories = caloriesPerServing x portion
5. Kiểm tra hợp lệ -> lưu bằng coroutine Dispatchers.IO -> hiện Snackbar

ui/meal/AddEditMealDialog.kt · saveMeal()

```
val quick = binding.cbQuickEntry.isChecked
val name = if (quick) binding.etQuickName.text
        ?.toString()?.trim().orEmpty()
    else selectedFood?.name.orEmpty()
if (name.isEmpty()) { /* báo lỗi tại ô nhập */ return }
val portion = parsePortion()   // "1,5" -> 1.5
if (portion <= 0) { setPortionInvalid(true); return }
val entry = MealEntry(
    id = old?.id ?: 0L,
    foodId = if (quick) null else selectedFood?.id,
    foodName = name,
    caloriesPerServing = calPerServing,
    portion = portion,
    totalCalories = calPerServing * portion,
    mealType = currentMealType(),
    date = selectedDate, time = selectedTime, ...)
lifecycleScope.launch(Dispatchers.IO) {
    if (old != null) mealDao.update(entry)
    else mealDao.insert(entry)
}
```

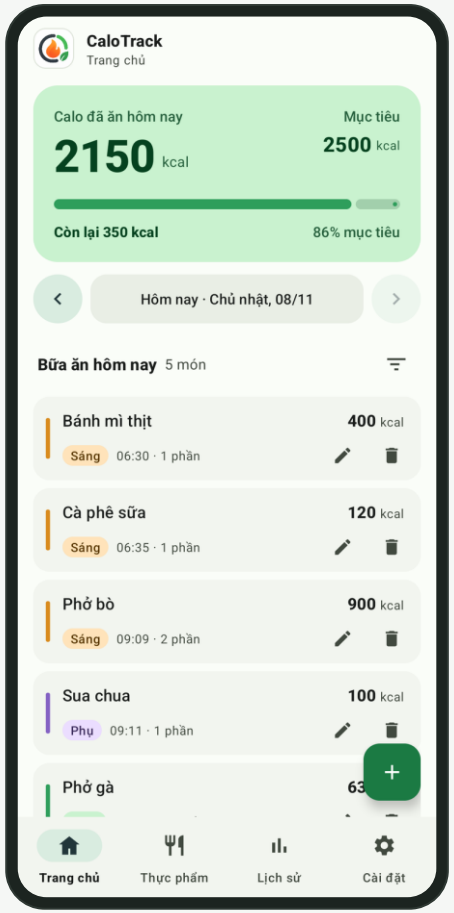


Tổng calo trong ngày và đổi màu theo mục tiêu

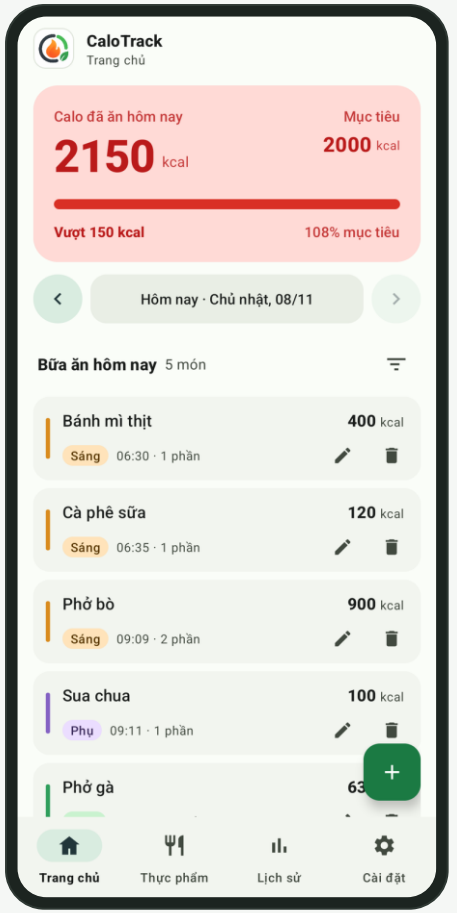
ui/home/HomeFragment.kt · updateOverview()

```
val safeTarget = target.coerceAtLeast(1)
val diff = safeTarget - eaten
val isOver = diff < 0
val pct = ((eaten * 100.0) / safeTarget).roundToInt()
tvRemain.text = if (!isOver)
    "Còn lại ${fmtCal(diff)} kcal"
else "Vượt ${fmtCal(-diff)} kcal"
binding.ringCalorieGauge.setProgress(pct, isOver)

// Đổi nền thẻ Hero, vòng tròn % và thanh tiến trình theo trạng thái:
// Xanh (bg_hero_under) khi dưới mục tiêu, Đỏ (bg_hero_over) khi vượt
binding.layoutHeroBg.setBackgroundResource(
    if (isOver) R.drawable.bg_hero_over
    else R.drawable.bg_hero_under
)
progressCal.setIndicatorColor(barColor)
progressCal.setProgressCompat(pct.coerceIn(0, 100), true)
```



Mục tiêu 2.500 -> Xanh



Mục tiêu 2.000 -> Đỏ

Lịch sử calo và biểu đồ cột 7 ngày

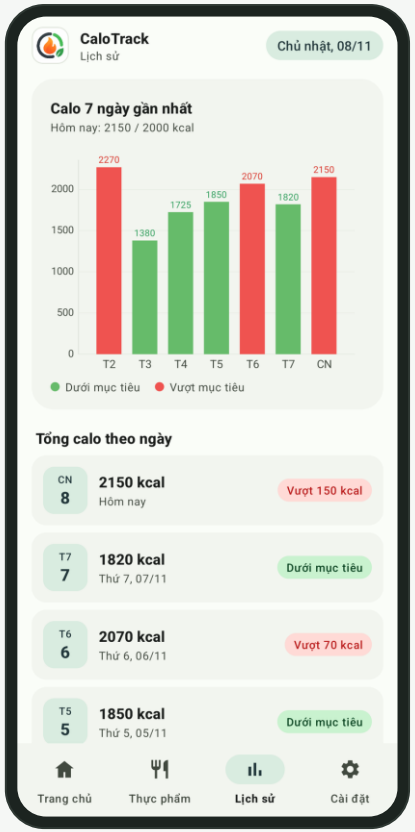
data/MealDao.kt + ui/history/HistoryViewModel.kt

```
@Query("SELECT date AS date,
  COALESCE(SUM(totalCalories),0) AS total
  FROM meals WHERE date BETWEEN :from AND :to
  GROUP BY date ORDER BY date ASC")
fun getDailyTotals(from: String, to: String)
    : Flow<List<DayTotal>>

val last7 = DateUtils.last7Days()
val weeklyTotals = mealDao.getDailyTotals(
    last7.first(), last7.last()).asLiveData()
```

ui/history/HistoryFragment.kt · updateChart()

```
val entries = days.mapIndexed { i, date ->
  BarEntry(i.toFloat(), (totalByDate[date] ?: 0.0).toFloat()) }
val overFlags = days.map { (totalByDate[it] ?: 0.0) > target }
dataSet.colors = overFlags.map { if (it) colorOver else colorUnder }
```



Đối chiếu kết quả SQL và biểu đồ

Ngày	SUM(totalCalories)	Trạng thái
T2 · 02/11	2.270 kcal	Vượt mục tiêu
T3 · 03/11	1.380 kcal	Dưới mục tiêu
T4 · 04/11	1.725 kcal	Dưới mục tiêu
T5 · 05/11	1.850 kcal	Dưới mục tiêu
T6 · 06/11	2.070 kcal	Vượt mục tiêu
T7 · 07/11	1.820 kcal	Dưới mục tiêu
CN · 08/11	2.150 kcal	Vượt mục tiêu

Những ngày chưa có bữa ăn được tự động điền giá trị 0 để biểu đồ luôn hiển thị đủ 7 cột. Màu cột và nhãn số tự động đổi Xanh / Đỏ so với mức mục tiêu 2.000 kcal.

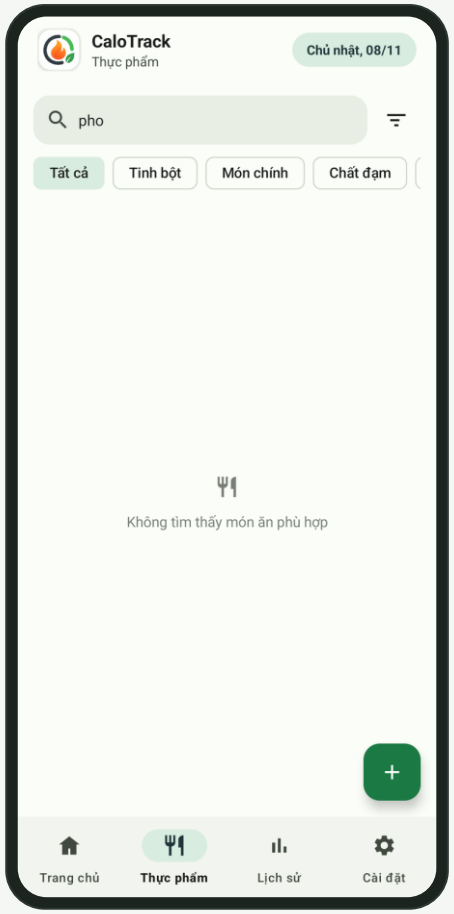
Tìm kiếm không dấu, bộ lọc và xóa kèm Hoàn tác

ui/food/FoodListFragment.kt · Tìm kiếm tiếng Việt không dấu

```
private fun normalize(s: String): String {
    val nfd = Normalizer.normalize(s.lowercase(), Normalizer.Form.NFD)
    return nfd.replace(Regex("\\p{InCombiningDiacriticalMarks}+"), "")
              .replace('đ', 'd')
}
// Gõ "pho" khớp ngay "Phở bò", "Phở gà", "Phở cuốn"
if (q.isNotEmpty()) {
    list = list.filter { normalize(it.name).contains(q) }
}
```

ui/food/FoodListFragment.kt · Xóa thực phẩm kèm Snackbar Hoàn tác

```
private fun deleteFoodWithUndo(food: Food) {
    viewLifecycleOwner.lifecycleScope.launch {
        withContext(Dispatchers.IO) { foodDao.delete(food) }
        Snackbar.make(binding.root, R.string.food_deleted, LENGTH_LONG)
            .setAction(R.string.undo) {
                lifecycleScope.launch(Dispatchers.IO) { foodDao.insert(food) }
            }.show()
    }
}
```



Gõ 'pho' -> ra 3 món Phở



Xóa + Hoàn tác 1 chạm

Tổng quan lưu trữ: SQLite (calotrack.db) & SharedPreferences

Bảng foods (SQLite)

```
id (INTEGER PK AUTOINCREMENT)
name (TEXT NOT NULL)
caloriesPerServing (REAL)
servingUnit · category (TEXT)
```

89 món ăn nạp sẵn thuộc 8 nhóm

Bảng meals (SQLite)

```
id (INTEGER PK AUTOINCREMENT)
foodId (INTEGER NULLABLE)
foodName · caloriesPerServing
portion · totalCalories · mealType
date (yyyy-MM-dd) · time (HH:mm)
```

Nhật ký bữa ăn theo ngày & giờ

calotrack_prefs.xml

```
target_calories (Int, mặc định 2000)
dark_mode (Boolean, mặc định false)
unit (String, mặc định "kcal")
demo_seeded (Boolean)
```

Lưu cặp khóa - giá trị SharedPreferences

Room ORM sinh bảng tự động · KSP kiểm tra kiểu dữ liệu và sinh mã tạo bảng SQLite từ các lớp @Entity ngay lúc biên dịch.

Lưu bản sao (Snapshot) trong meals · meals lưu trực tiếp foodName và caloriesPerServing để khi sửa/xóa thực phẩm không làm sai lịch sử cũ.

Chuỗi ngày yyyy-MM-dd & giờ HH:mm · Cho phép so sánh chuỗi, truy vấn BETWEEN và sắp xếp ORDER BY time ASC chính xác tuyệt đối.

Khởi tạo 89 món trong onCreate · Nạp đồng bộ toàn bộ 89 món ăn ngay trong transaction tạo file calotrack.db lần đầu tiên.

Sơ đồ quan hệ thực thể (ERD: 2 bảng SQLite)

foods

@Entity(tableName = "foods") · 89 món ăn cơ bản

PK	id	INTEGER	Khóa chính tự tăng
	name	TEXT	Tên món ăn (NOT NULL)
	caloriesPerServing	REAL	Calo trên 1 khẩu phần
	servingUnit	TEXT	Đơn vị (bát, tô, đĩa, ổ...)
	category	TEXT	Nhóm món ăn (8 nhóm)

1 ---- 0..N

foods.id -> meals.foodId
(Tham chiếu logic)

meals

@Entity(tableName = "meals") · Lịch sử bữa ăn

PK	id	INTEGER	Khóa chính tự tăng
FK	foodId	INTEGER	NULL nếu nhập nhanh
	foodName	TEXT	Bản sao tên món ăn
	caloriesPerServing	REAL	Bản sao calo / khẩu phần
	portion	REAL	Số khẩu phần (0,5 · 1,0...)
	totalCalories	REAL	caloriesPerServing * portion
	mealType	TEXT	BREAKFAST / LUNCH / DINNER / SNACK
	date	TEXT	Ngày ăn (yyyy-MM-dd)
	time	TEXT	Giờ ăn (HH:mm)
	createdAt	INTEGER	Thời điểm tạo (ms)

Vì sao CSDL SQLite chỉ cần 2 bảng?

- **Gọn nhẹ cho thiết bị di động:** Ứng dụng chạy độc lập trên máy người dùng nên chỉ cần 2 bảng cốt lõi (**foods** và **meals**).
- **Bảo toàn lịch sử bữa ăn:** Bảng **meals** lưu bản sao **foodName** và **caloriesPerServing** tại thời điểm ăn, giúp việc sửa/xóa món trong **foods** không làm sai lệch lịch sử.
- **Hỗ trợ Nhập nhanh linh hoạt:** Khi người dùng nhập nhanh món ngoài danh mục, **foodId** nhận giá trị **NULL**.

Các câu truy vấn SQL trong Room DAO

SELECT · Bữa ăn trong ngày & tổng calo · MealDao.kt

```
SELECT * FROM meals
WHERE date = :date ORDER BY time ASC

SELECT COALESCE(SUM(totalCalories), 0)
FROM meals WHERE date = :date
```

GROUP BY · Tổng calo từng ngày (Lịch sử) · MealDao.kt

```
SELECT date AS date,
  COALESCE(SUM(totalCalories), 0) AS total
FROM meals
GROUP BY date ORDER BY date DESC
```

SELECT · Danh mục & tìm kiếm thực phẩm · FoodDao.kt

```
SELECT * FROM foods                -- getAllFoods()
ORDER BY name COLLATE NOCASE ASC

SELECT * FROM foods WHERE name     -- searchFoods(q)
LIKE '%' || :q || '%'
```

INSERT · UPDATE · DELETE · Room KSP tự sinh mã

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insert(food: Food): Long
@Update suspend fun update(meal: MealEntry)
@Delete suspend fun delete(meal: MealEntry)
```

Khởi tạo CSDL và lưu thiết lập bằng SharedPreferences

data/AppDatabase.kt · getInstance()

```
Room.databaseBuilder(appContext,
        AppDatabase::class.java, "calotrack.db")
    .addCallback(object : RoomDatabase.Callback() {
        // Nạp đồng bộ 89 món trong transaction tạo DB
        override fun onCreate(db: SupportSQLiteDatabase) {
            super.onCreate(db)
            SeedData.getFoods().forEach { food ->
                db.insert("foods", CONFLICT_NONE,
                    ContentValues().apply {
                        put("name", food.name)
                        put("caloriesPerServing", ...)
                        put("servingUnit", food.servingUnit)
                        put("category", food.category)
                    })
            }
        }
    })
    .build()
```

util/PrefsManager.kt

```
private val prefs = context.getSharedPreferences(
    "calotrack_prefs", Context.MODE_PRIVATE)
var targetCalories: Int
    get() = prefs.getInt(KEY_TARGET_CALORIES, 2000)
    set(v) = prefs.edit().putInt(KEY_TARGET_CALORIES, v).apply()
var darkMode: Boolean   // mặc định false
val unit: String get() = prefs.getString(KEY_UNIT, "kcal")...
```

ui/settings/SettingsFragment.kt

```
val value = etTarget.text?.toString()?.trim()?.toIntOrNull()
if (value != null && value in 500..10000) {
    prefs.targetCalories = value
    homeViewModel.refreshTarget() // đồng bộ sang Trang chủ & Lịch sử
} else { /* Toast "Vui lòng nhập từ 500 đến 10.000 kcal" */ }
```

Quy trình kiểm thử và bảng ca kiểm thử

1 · Biên dịch
gradlew assembleDebug

2 · Máy thật
Redmi 10C & Note 13 Pro 5G

3 · Máy ảo
Android Emulator · cài mới

4 · Đối chiếu DB
Đọc calotrack.db & prefs qua ADB

Ca kiểm thử	Kỳ vọng	Kết quả thực tế
Mở ứng dụng lần đầu sau khi cài mới	Bảng foods có sẵn 89 món	ĐẠT · 89 món thuộc 8 nhóm
Thêm Phở bò (450 kcal), khẩu phần 1,5	Tự tính 675 kcal	ĐẠT · 675 kcal · tổng ngày 520 -> 1.195
Sửa bữa Phở bò: khẩu phần 1,5 -> 2,0	Cập nhật cùng id lên 900 kcal	ĐẠT · id=78 cập nhật thành 900 kcal
Nhập nhanh 'Sua chua' 100 kcal (Phụ)	Lưu bữa ăn với foodId = NULL	ĐẠT · foodId=NULL · 100 kcal
Bấm Lưu khi chưa chọn thực phẩm	Báo lỗi tại ô nhập, không lưu	ĐẠT · Hiện lỗi · không sinh dòng mới
Tổng ngày 2.150 kcal > mục tiêu 2.000	Thẻ Hero đổi màu Đỏ, báo Vượt	ĐẠT · Thẻ đỏ · 'Vượt 150 kcal' (108%)
Đặt mục tiêu 300 rồi đặt 2.500	Từ chối 300, lưu 2.500 vào prefs	ĐẠT · Toast nhắc lỗi · target_calories = 2500
Tìm 'pho' không dấu ở tab Thực phẩm	Khớp các món có chữ 'Phở'	ĐẠT · 3 món: Phở bò, Phở cuốn, Phở gà
Xóa món ăn rồi bấm Hoàn tác (Undo)	Khôi phục đúng bản ghi & id	ĐẠT · 89 -> 88 -> 89 dòng (giữ nguyên id)
So sánh biểu đồ 7 ngày với SQL SUM	Khớp tổng calo cả 7 ngày	ĐẠT · 7/7 ngày khớp hoàn toàn
Bật giao diện tối, đóng rồi mở lại app	Giữ nguyên dark_mode = true	ĐẠT · Mở lên ở chế độ tối mượt mà
Đổi mục tiêu ở Cài đặt -> về Trang chủ	Cập nhật ngay mục tiêu mới	ĐẠT · Cập nhật tức thì (2.500 kcal · 86%)

Kết quả đạt được

9 / 9

yêu cầu chức năng bắt buộc của Đề 10 hoàn thành

12 / 12

ca kiểm thử thực tế đạt 100% trên phần cứng thật

89 Ảnh

ảnh chụp món ăn thật đóng gói sẵn kèm bộ đệm LruCache

7,3 MB

dung lượng file APK hoàn chỉnh kèm đồ họa 3D Soft-Tech



Mã nguồn: github.com/ndtien0406-design/app_calo · Đã kiểm thử thực tế trên Xiaomi Redmi 10C (720x1650) và Xiaomi Redmi Note 13 Pro 5G (1220x2712 @ 480dpi).

Khó khăn và giải pháp kỹ thuật

Danh sách món có thể trống ở lần mở đầu tiên	Nạp đồng bộ 89 món ngay trong RoomDatabase.Callback.onCreate cùng transaction khởi tạo file SQLite
Chuyển tab bằng Fragment hide/show không gọi lại onResume	Chia sẻ HomeViewModel qua activityViewModels() và bắt sự kiện onHiddenChanged() để đồng bộ mục tiêu tức thì
Tìm kiếm tiếng Việt không dấu và chuyển ngày qua nửa đêm	Chuẩn hóa chuỗi Unicode NFD tách dấu kết hợp tự động kiểm tra sang ngày mới khi quay lại Trang chủ
Sửa hoặc xóa thực phẩm làm sai lệch lịch sử bữa ăn cũ	Lưu bản sao foodName, caloriesPerServing trực tiếp vào bảng meals và cho phép foodId nhận NULL
Nút nổi (+) đè lên thông tin thẻ cuối danh sách khi cuộn	Xây dựng UiverseOrbFabView tự động thu gọn né thẻ khi cuộn, đệm đáy 112dp, hào quang Sonar và xoay dấu + 135 độ

Hạn chế và hướng phát triển

Hạn chế hiện tại

- Đơn vị năng lượng đang cố định ở mức kcal (hiển thị chỉ đọc trong Cài đặt), chưa có bộ quy đổi sang kJ
- Chỉ mới theo dõi tổng năng lượng (calo), chưa tách chi tiết 3 nhóm chất đa lượng (Đạm · Béo · Tinh bột)
- Dữ liệu lưu cục bộ trên máy, chưa có tính năng xuất/nhập file sao lưu JSON/CSV hoặc đồng bộ đám mây
- CSDL Room đang ở version 1, chưa viết kịch bản Migration tự động khi nâng cấp cấu trúc bảng

Hướng phát triển

- Bổ sung tùy chọn chuyển đổi đơn vị kcal / kJ (1 kcal = 4,184 kJ) tận dụng khóa unit sẵn có trong PrefsManager
- Mở rộng bảng foods và meals để thống kê thêm Protein, Carb, Fat và lượng nước uống hằng ngày
- Hỗ trợ sao lưu / khôi phục dữ liệu ra file JSON hoặc đồng bộ tự động qua tài khoản Google Drive
- Bổ sung bộ kiểm thử tự động Room in-memory cho tầng DAO và kịch bản Espresso cho luồng giao diện

Tổng kết & Hỏi đáp

CaloTrack là ứng dụng Android **hoàn chỉnh và vận hành thực tế trên điện thoại**: đáp ứng trọn vẹn 9 yêu cầu của Đề 10, thiết kế CSDL SQLite 2 bảng tối giản kết hợp SharedPreferences và giao diện Material 3 3D.

Kỹ thuật

Kotlin, Room SQLite, MVVM LiveData/Flow, Coroutines và Custom Canvas 60fps

Sản phẩm

89 món Việt kèm ảnh thật, tự tính calo, biểu đồ 7 ngày, giao diện Sáng/Tối

Bài học

Lưu bản sao bảo toàn lịch sử bữa ăn, luồng UI phản ứng, kiểm thử trên máy thật

Hỏi & Đáp · Xin cảm ơn!

Cảm ơn thầy cô và các bạn đã theo dõi phần trình bày!

Nhóm [SỐ NHÓM] · CaloTrack (Đề 10) · github.com/ndtien0406-design/app_calo